MEDICHECK

# PROYECTO INTERMODULAR

## *MEDICHECK FASE II - Manual Técnico*

Miguel González Jiménez

Instituto I.E.S. Albarregas - Desarrollo de Aplicaciones Multiplataformas

Tutor: María Mercedes Martinez Fragoso

0492: Proyecto Intermodular

28 de enero de 2026

# Índice

**1. Introducción** ..... **3**
**2. Descripción de la demo funcional** ..... **3**
**3. Arquitectura del sistema** ..... **4**
**4. Tecnologías utilizadas** ..... **6**
4.1 Firebase y persistencia de datos ..... 8
**5. Estructura del proyecto** ..... **8**
**6. Funcionalidades implementadas** ..... **12**
6.1 Lenguaje de programación ..... 12
6.2 Framework de desarrollo ..... 13
6.3 Gestión del estado ..... 13
6.4 Control de versiones ..... 14
6.5 Herramientas de desarrollo ..... 15
6.6 Integración clínica con OpenFDA ..... 15
6.7 Sistema de notificaciones ..... 16
**7. Flujo de funcionamiento de la aplicación** ..... **16**
7.1 Sistema de autenticación ..... 16
7.2 Dashboard principal ..... 17
7.3 Gestón de pacientes ..... 18
7.4 Gestión de medicaciones ..... 19
7.5 Gestión de citas médicas ..... 20
7.6 Agenda general de citas ..... 21
7.7 Sistema de emergencia ..... 21
7.8 Sistema de navegación ..... 22
**8. Pruebas realizadas** ..... **22**
8.1 Estado actual del desarrollo ..... 23
8.2 Limitaciones actuales ..... 24
8.3 Próximas mejoras previstas ..... 24

## 1. Introducción

MediCheck es una aplicación móvil desarrollada como proyecto intermodular del ciclo de Desarrollo de Aplicaciones Multiplataforma.

El objetivo principal de la aplicación es facilitar el seguimiento de medicaciones y tratamientos médicos de personas dependientes, permitiendo a cuidadores, familiares o personal sanitario gestionar de forma sencilla la información relacionada con pacientes, tratamientos y citas médicas.

En muchos casos, la gestión de medicaciones puede resultar compleja, especialmente cuando una persona depende de varios cuidadores o cuando existen múltiples tratamientos con diferentes horarios y dosis

MediCheck propone una solución que centraliza toda esta información en una única aplicación accesible desde dispositivos móviles.

La aplicación permite registrar pacientes, añadir mediaciones, gestionar citas médicas y consultar la información relevante desde un panel principal diseñado para facilitar el acceso rápido a los datos más importantes.

## 2. Descripción de la demo funcional

Durante esta fase del proyecto, MediCheck ha evolucionado desde una demo funcional hacia un MVP completamente operativo conectado a servicios reales de Firebase.

La aplicación permite recorrer el flujo completo del sistema desde la autenticación hasta la gestión avanzada de pacientes, medicaciones y citas médicas, incorporando además validaciones clínicas, sincronización en tiempo real y sistemas de notificaciones.

La arquitectura actual implementa persistencia real de datos mediante Firebase Auth, Cloud Firestore y Firebase Storage, permitiendo sincronización automática entre dispositivos y sesiones persistentes de usuario.

La versión actual de la demo permite:

- Sistema completo de autenticación mediante Firebase Auth.
- Gestión de roles diferenciados (Administrador y Cuidador)
- CRUD completo de pacientes.
- CRUD completo de medicaciones.
- CRUD completo de citas médicas.
- Agenda global de citas sincronizada en tiempo real.
- Sistema de notificaciones locales y Push Notifications mediante Firebase Cloud Messaging.
- Validación clínica mediante integración con OpenFDA.
- Control de stock y seguimiento de tratamientos.
- Dashboard reactivo con información médica relevante.
- Sistema de emergencia SOS.
- Sincronización automática mediante arquitectura Provider + Streams.
- Persistencia completa en la nube mediante Firestore.

Estas funcionalidades permiten demostrar un flujo end-to-end completamente funcional y validan tanto la arquitectura técnica como la viabilidad real del proyecto.

## 3. Arquitectura del sistema

La aplicación MediCheck ha sido desarrollada siguiendo una arquitectura modular basada en la separación de responsabilidades. Este enfoque permite mantener el código organizado, facilitar su mantenimiento y permitir la ampliación futura del sistema.

La arquitectura del proyecto se divide en diferentes capas, cada una con una responsabilidad específica dentro del sistema.

Las principales capas de la arquitectura son:

**Models**

Los modelos definen la estructura de los datos utilizados en la aplicación. Representan las entidades principales del sistema como pacientes, medicaciones o citas médicas.

En MediCheck se han implementado los siguientes modelos principales:

- Patient

- Medication
- Appointment

**Providers**

Los providers gestionan el estado global de la aplicación (Y algunas lógicas por ahora) utilizando el patrón de gestión de estado Provider.

Gracias a este sistema, los datos pueden compartirse entre distintas pantallas y actualizarse automáticamente cuando se producen cambios en el sistema.

Los providers principales implementados son:

- PatientProvider
- MedicationProvider
- AppointmentProvider

**Services**

Los servicios se encargan de gestionar la lógica de acceso a datos y la comunicación con posibles servicios externos o Apis.

Actualmente estos servicios permiten conectar la aplicación con un backend o una base de datos externa.

**Screens**

Las pantallas representan la interfaz principal de la aplicación y contienen la lógica visual de cada sección del sistema.

Las pantallas están organizadas en módulos según su funcionalidad:

- Autenticación.
- Home o dashboard principal.
- Gestión de pacientes.
- Gestión de medicaciones.
- Gestión de citas médicas.
- Emergencias.
- Configuración.

**Widgets**

Los widgets contienen componentes reutilizables utilizados en diferentes partes de la aplicación.

Entre los widgets reutilizables implementados se encuentran:

- Tarjetas de pacientes
- Tarjetas de citas médicas
- Botones reutilizables
- Campos de entrada de formularios
- Layout global de la aplicación

Esta arquitectura modular permite mantener el proyecto organizado y facilita la incorporación de nuevas funcionalidades en futuras versiones de la aplicación.

**Seguridad y control de acceso**

MediCheck implementa un sistema de control de acceso basado en roles (RBAC), permitiendo diferenciar entre usuarios Administradores y Cuidadores.

El Administrador dispone de permisos globales para gestionar pacientes y asignar cuidadores, mientras que los Cuidadores únicamente pueden acceder a la información de los pacientes bajo su supervisión.

La aplicación utiliza Firebase Authentication para gestionar las sesiones de usuario y Cloud Firestore para filtrar el acceso a los datos mediante identificadores asociados al cuidador.

Este sistema permite mejorar la privacidad y garantizar que cada usuario únicamente acceda a la información médica autorizada.

## 4. Tecnologías utilizadas

El desarrollo de MediCheck se ha realizado utilizando distintas tecnologías y herramientas que permiten crear aplicaciones móviles modernas, escalables y mantenibles.

**Flutter**

Flutter es el framework principal utilizado para el desarrollo de la aplicación.

Se trata de un framework desarrollado por Google que permite crear aplicaciones multiplataforma utilizando el lenguaje de programación Dart.

Flutter ofrece un sistema de widgets muy flexible que facilita la creación de interfaces modernas y reutilizables.

**Dart**

Dart es el lenguaje de programación utilizado por Flutter.

Permite desarrollar aplicaciones de alto rendimiento con una sintaxis clara y orientada a objetos.

**Provider**

Para la gestión del estado de la aplicación se ha utilizado la librería Provider.

Este patrón permite compartir datos entre distintas pantallas de la aplicación y actualizar automáticamente la interfaz cuando los datos cambian.

Gracias a Provider la aplicación mantiene sincronizados los datos entre los distintos módulos del sistema.

**Git y GitHub**

El control de versiones del proyecto se realiza mediante Git.

Aunque empecé a hacerlo tarde el repositorio del proyecto se encuentra alojado en GitHub, lo que permite mantener un historial de cambios del desarrollo y facilitar la colaboración y revisión del código.

**Visual Studio Code**

El entorno de desarrollo utilizado para programar la aplicación ha sido Visual Studio Code, que ofrece integración con Flutter y herramientas de depuración que facilitan el desarrollo de aplicaciones móviles.

**Figma**

Figma se ha utilizado para el diseño inicial del prototipo de la interfaz de usuario de MediCheck.

Este prototipo permitió definir las pantallas principales antes de comenzar el desarrollo de la aplicación.

### 4.1 Firebase y persistencia de datos.

MediCheck utiliza Firebase como backend principal de la aplicación.

Los servicios integrados son:

- Firebase Authentication: Gestiona el registro, inicio de sesión y persistencia de usuarios dentro del sistema.
- Cloud Firestore: Se utiliza como base de datos NoSQL en tiempo real para almacenar pacientes, medicaciones, citas médicas, perfiles de usuario y relaciones entre cuidadores y pacientes.

Gracias a Firestore, los cambios realizados desde un dispositivo se reflejan automáticamente en el resto de sesiones activas.

Firebase Storage. Permite almacenar imágenes asociadas a perfiles de usuario o pacientes

Firebase Cloud Messaging (FCM). Sistema utilizado para el envío de notificaciones push relacionadas con citas médicas y avisos importantes del sistema.

La integración de Firebase permite disponer de un backend escalable, seguro y sincronizado en tiempo real sin necesidad de implementar un servidor propio.

## 5. Estructura del proyecto

El proyecto MediCheck está organizado siguiendo una estructura modular que separa los distintos componentes de la aplicación según su responsabilidad.

Esta organización permite mantener el código limpio, facilitar su mantenimiento y permitir futuras ampliaciones del sistema, la estructura principal del proyecto es la siguiente:

```
lib/
├── main.dart
├── firebase_options.dart        # Configuración generada de Firebase
│
├── routes/
│   └── app_routes.dart          # Gestor de rutas de la aplicación
│
├── models/                      # Objetos de datos
│   ├── patient.dart
│   ├── medication.dart
│   ├── medication_log.dart      # Historial de tomas
│   ├── appointment.dart
│   └── incidence.dart           # Modelo para incidencias/reportes
│
├── services/                    # Lógica de negocio y APIs
│   ├── auth_service.dart        # Gestión de Firebase Auth
│   ├── user_service.dart        # Perfiles de usuario (Admin/Cuidador)
│   ├── patient_service.dart     # CRUD de pacientes en Firestore
│   ├── medication_service.dart  # Gestión de tratamientos
│   ├── fda_service.dart         # Conexión con OpenFDA API
│   ├── appointment_service.dart # Gestión de citas médicas
│   ├── notification_service.dart # Recordatorios locales y Push (FCM)
│   ├── maps_service.dart        # Lógica para SOS y geolocalización
│   ├── incidence_service.dart   # Gestión de reportes de errores
│   ├── storage_service.dart     # Subida de imágenes a Firebase Storage
│   └── fhir_service.dart        # Adaptador de estándares médicos
│
```

```
├── providers/              # Gestión de estado (Provider)
│   ├── auth_provider.dart
│   ├── patient_provider.dart
│   ├── medication_provider.dart
│   └── appointment_provider.dart
│
├── screens/                # Vistas de la aplicación
│   ├── auth/               # Autenticación
│   │   ├── splash_screen.dart
│   │   ├── login_screen.dart
│   │   ├── register_screen.dart
│   │   ├── recover_password_screen.dart
│   │   └── terms_screen.dart
│   │
│   ├── admin/              # Panel exclusivo Administrador
│   │   ├── admin_dashboard_screen.dart
│   │   ├── caregivers_list_screen.dart
│   │   └── incidences_screen.dart
│   │
│   ├── home/               # Dashboard Cuidador
│   │   └── home_screen.dart
│   │
│   ├── patients/           # Gestión de Pacientes
│   │   ├── patients_list_screen.dart
│   │   ├── patient_detail_screen.dart
│   │   ├── add_patient_screen.dart
│   │   └── edit_patient_screen.dart
```

```
│   │
│   ├── medications/          # Gestión de Medicación
│   │   ├── medications_list_screen.dart
│   │   ├── add_medication_screen.dart
│   │   └── edit_medication_screen.dart
│   │
│   ├── appointments/         # Agenda y Citas
│   │   ├── appointments_screen.dart
│   │   ├── appointment_detail_screen.dart
│   │   ├── add_appointment_screen.dart
│   │   ├── edit_appointment_screen.dart
│   │   └── agenda_screen.dart
│   │
│   ├── emergency/            # Protocolo SOS
│   │   └── emergency_screen.dart
│   │
│   └── settings/             # Ajustes de perfil
│       └── settings_screen.dart
│
└── widgets/                  # Componentes reutilizables
    ├── common/               # Layout y navegación
    │   ├── app_header.dart
    │   ├── app_bottom_nav.dart
    │   ├── app_layout.dart
    │   └── profile_menu.dart
    │
    ├── cards/                # Tarjetas informativas
```

```
│   ├── patient_card.dart
│   ├── medication_card.dart
│   └── appointment_card.dart
│
├── forms/                # Elementos de formulario
│   ├── text_input.dart
│   ├── date_input.dart
│   ├── dropdown_input.dart
│   └── primary_button.dart
│
└── inputs/               # Widgets de entrada específicos
    ├── password_input.dart
    └── text_input.dart
```

Aunque pueda resultar un poco extenso, es más corto de lo que parece.

## 6. Funcionalidades implementadas

El desarrollo de la aplicación MediCheck se ha realizado utilizando tecnologías modernas orientadas al desarrollo de aplicaciones móviles multiplataforma.

Estas tecnologías permiten construir una aplicación modular, mantenible y preparada para futuras ampliaciones.

### 6.1 Lenguaje de programación

El desarrollo de la aplicación se ha realizado utilizando Dart, que es el lenguaje de programación oficial utilizado por Flutter.

Dart está diseñado para desarrollar aplicaciones modernas con alto rendimiento y permite estructurar el código de forma clara mediante clases, widgets y gestión de estado.

Su integración directa con Flutter facilita el desarrollo de interfaces dinámicas y reactivas.

## 6.2 Framework de desarrollo

El framework utilizado para el desarrollo de MediCheck es Flutter, un framework de código abierto desarrollado por Google para la creación de aplicaciones móviles multiplataforma.

Flutter permite desarrollar aplicaciones utilizando un único código base que puede ejecutarse en diferentes sistemas operativos.

Entre sus principales ventajas se encuentran:

- Desarrollo rápido mediante sistema de Hot Reload.

- Interfaz basada en widgets reutilizables.

- Alto rendimiento al compilar directamente a código nativo.

Gracias a Flutter se ha podido construir una interfaz visual clara y organizada para facilitar el uso de la aplicación por parte de cuidadores y personal sanitario.

## 6.3 Gestión del estado

Para gestionar el estado de la aplicación se ha utilizado el patrón Provider.

Provider permite compartir datos entre diferentes partes de la aplicación y actualizar automáticamente las pantallas cuando la información cambia.

En MediCheck se utilizan distintos providers para gestionar la información del sistema, como por ejemplo:

- PatientProvider, encargado de gestionar los pacientes registrados.

- MedicationProvider, encargado de gestionar las medicaciones.

- AppointmentProvider, encargado de gestionar las citas médicas.

Este sistema permite mantener sincronizados los datos entre las diferentes pantallas del sistema

Además, la aplicación utiliza una arquitectura reactiva basada en Streams y Provider, permitiendo que los cambios realizados sobre pacientes, citas o medicaciones se actualicen automáticamente en tiempo real en todas las pantallas de la aplicación sin necesidad de recargar manualmente la interfaz.

Este enfoque mejora la experiencia de usuario y permite mantener sincronizados los datos entre diferentes dispositivos conectados al sistema mediante Firebase Cloud Firestore.

## 6.4 Control de versiones

El proyecto utiliza Git como sistema de control de versiones.

Git permite mantener un historial de cambios del proyecto, facilitando el seguimiento del desarrollo y la recuperación de versiones anteriores si fuera necesario.

El código fuente del proyecto se encuentra almacenado en un repositorio de GitHub, lo que permite centralizar el desarrollo y compartir el proyecto con otros miembros del equipo o evaluadores.

## 6.5 Herramientas de desarrollo

Durante el desarrollo de MediCheck se han utilizado diferentes herramientas de desarrollo que han facilitado la implementación del proyecto.

Entre las principales herramientas utilizadas se encuentran:

- Visual Studio Code como entorno de desarrollo principal.
- Android Studio para el uso del emulador de Android y herramientas del SDK.
- GitHub Desktop para la gestión del repositorio.
- Trello para la organización de tareas del proyecto.

Estas herramientas han permitido mantener un desarrollo organizado y facilitar la gestión del proyecto durante todo el proceso de implementación.

## 6.6 Integración clínica con OpenFDA

Una de las funcionalidades más importantes implementadas en MediCheck es la integración con la Api oficial OpenFDA.

Esta integración permite consultar información real sobre medicamentos y principios activos utilizados en tratamientos médicos.

Cuando un cuidador registra una nueva medicación para un paciente, la aplicación analiza automáticamente el principio activo asociado al medicamento.

Si el sistema detecta que el paciente ya dispone de otro tratamiento compatible con el mismo principio activo en fechas similares, MediCheck genera una alerta preventiva indicando un posible riesgo de duplicidad o sobredosis.

Este sistema aporta una capa adicional de seguridad clínica y diferencia la aplicación de un simple gestor CRUD tradicional

### 6.7 Sistema de notificaciones

MediCheck incorpora un sistema híbrido de notificaciones y Push Notifications orientado a mejorar el seguimiento de tratamientos y citas médicas.

Las notificaciones locales permiten generar recordatorios incluso sin conexión a internet, avisando automáticamente sobre próximas tomas de medicación o citas médicas programadas.

Además, la aplicación utiliza Firebase Cloud Messaging (FCM) para el envío de notificaciones push desde el backend, permitiendo mostrar avisos importantes en tiempo real.

El sistema de notificaciones se integra directamente con la lógica de citas y tratamientos, permitiendo generar alertas automáticas y mejorar la experiencia de seguimiento médico del usuario.

## 7. Flujo de funcionamiento de la aplicación

La versión actual de MediCheck incluye una demo funcional que permite recorrer el flujo principal de la aplicación y utilizar las funcionalidades básicas del sistema.

El objetivo de esta demo es demostrar el funcionamiento general de la aplicación y validar la arquitectura del proyecto.

### 7.1 Sistema de autenticación

La aplicación incluye un sistema básico de autenticación que permite gestionar el acceso de los usuarios al sistema.

Las pantallas implementadas son:

- SplashScreen

- LoginScreen

- RegisterScreen

- RecoverPasswordScreen

Estas pantallas permiten:

- iniciar sesión en la aplicación

- registrar nuevos usuarios

- recuperar la contraseña de acceso

Una vez autenticado, el usuario accede al dashboard principal de la aplicación.

## 7.2 Dashboard principal

El HomeScreen funciona como panel principal del sistema.

Desde esta pantalla el usuario puede acceder rápidamente a las funcionalidades principales de la aplicación.

El dashboard incluye:

- accesos rápidos a pacientes

- acceso a citas médicas

- acceso al sistema de emergencia

- visualización de citas médicas del día

- visualización de medicaciones próximas

Este panel permite consultar de forma rápida la información más relevante del sistema.

## 7.3 Gestón de pacientes

La aplicación incluye un sistema completo de gestión de pacientes.

Las pantallas implementadas son:

- lista de pacientes

- añadir paciente

- editar paciente

- detalle de paciente

Las funcionalidades disponibles incluyen:

- visualizar lista de pacientes

- crear nuevos pacientes

- editar información del paciente

- eliminar pacientes

- acceder a la ficha detallada de cada paciente

Desde la ficha del paciente también es posible acceder a las medicaciones y citas médicas asociadas.

MediCheck también incorpora un sistema avanzado de transferencia de pacientes entre cuidadores.

Esta funcionalidad permite reasignar un paciente a otro cuidador registrado dentro del sistema manteniendo toda su información médica asociada.

La operación se realiza de forma atómica mediante Firebase Firestore, garantizando que los datos de medicaciones, citas médicas y relaciones del paciente se mantengan consistentes durante todo el proceso.

Este sistema evita pérdidas de información y mejora la gestión colaborativa entre cuidadores.

## 7.4 Gestión de medicaciones

MediCheck permite gestionar los tratamientos médicos asociados a cada paciente.

Las pantallas implementadas son:

- lista de medicaciones
- añadir medicación
- editar medicación

Cada medicación incluye información como:

- nombre del medicamento
- dosis
- tipo de medicamento
- frecuencia
- fecha de inicio
- fecha de finalización

La aplicación también implementa un sistema básico de control de stock de tratamientos.

Cada vez que una medicación es administrada y confirmada por el cuidador, el sistema descuenta automáticamente las unidades disponibles asociadas al tratamiento.

Este control permite mejorar el seguimiento del consumo de medicamentos y prevenir posibles errores relacionados con la falta de stock.

## 7.5 Gestión de citas médicas

La aplicación permite registrar y gestionar citas médicas de los pacientes.

Las pantallas implementadas incluyen:

- lista de citas médicas

- añadir cita médica

- editar cita médica

Cada cita médica contiene información como:

- paciente

- lugar de la cita

- fecha

- hora

La gestión de las citas se realiza mediante el AppointmentProvider, que se encarga de almacenar y actualizar la información dentro de la aplicación.

## 7.6 Agenda general de citas

Durante el desarrollo de la demo se ha implementado una pantalla adicional denominada AgendaScreen.

Esta pantalla permite visualizar todas las citas médicas registradas en el sistema en una única lista ordenada por fecha.

Las funcionalidades de esta pantalla incluyen:

- visualizar todas las citas registradas

- ordenar las citas por fecha

- abrir la edición de una cita al pulsarla

Esta funcionalidad facilita la consulta global de las citas médicas del sistema.

## 7.7 Sistema de emergencia.

La aplicación incluye una pantalla de emergencia accesible desde el dashboard principal.

Esta funcionalidad está diseñada para permitir una actuación rápida en situaciones críticas.

Actualmente la pantalla permite:

- mostrar una alerta de emergencia

- confirmar la acción antes de ejecutarla

En futuras versiones esta funcionalidad podría integrarse con servicios externos de emergencia.

### 7.8 Sistema de navegación

La navegación de la aplicación se encuentra centralizada en el archivo routes/app_routes.dart.

Este sistema permite definir todas las rutas del sistema en un único lugar y facilita la navegación entre las diferentes pantallas de la aplicación.

Entre las rutas principales se encuentran:

- login
- home
- pacientes
- medicaciones
- citas médicas
- agenda de citas
- emergencia
- configuración

Esto permite mantener el proyecto organizado y evitar errores en la navegación.

## 8. Pruebas realizadas

La aplicación MediCheck se encuentra actualmente en una fase de demo funcional que permite demostrar el funcionamiento general del sistema.

El proyecto ya cuenta con una arquitectura modular completa y una navegación funcional entre las diferentes pantallas de la aplicación.

El sistema permite gestionar información básica relacionada con pacientes, medicaciones y citas médicas, proporcionando una base sólida para futuras ampliaciones.

Durante el desarrollo del proyecto se realizaron diferentes pruebas funcionales para validar el correcto funcionamiento de la aplicación y comprobar la estabilidad de los distintos módulos implementados.

Entre las principales pruebas realizadas se encuentran:

- Validación del sistema de autenticación mediante Firebase Auth.
- Registro y persistencia de usuarios.
- Creación, edición y eliminación de pacientes.
- Gestión completa de medicaciones y tratamientos.
- Registro y modificación de citas médicas.
- Sincronización automática de datos mediante Cloud Firestore.
- Verificación del sistema de roles y restricciones de acceso.
- Comprobación de alertas clínicas mediante OpenFDA.
- Validación de notificaciones locales y Push Notifications.
- Navegación completa entre pantallas y persistencia de sesión.

## 8.1 Estado actual del desarrollo

Actualmente la aplicación dispone de las siguientes funcionalidades implementadas:

- sistema de autenticación básica
- navegación completa entre pantallas
- dashboard funcional
- gestión de pacientes
- gestión de medicaciones
- gestión de citas médicas
- agenda general de citas
- pantalla de emergencia
- arquitectura modular organizada
- componentes reutilizables

La aplicación permite recorrer prácticamente todo el flujo principal del sistema y demostrar el funcionamiento general de MediCheck.

## 8.2 Limitaciones actuales

Aunque MediCheck dispone actualmente de un MVP funcional completo, todavía existen algunas limitaciones y funcionalidades previstas para futuras iteraciones.

Entre las principales limitaciones actuales se encuentran:

- El sistema SOS funciona como simulación y todavía no se encuentra conectado con servicios reales de emergencias.
- Algunas validaciones clínicas de OpenFDA dependen de la disponibilidad externa de la API.
- El sistema no dispone todavía de autenticación biométrica.
- La exportación de informes médicos en PDF todavía

## 8.3 Próximas mejoras previstas

Aunque MediCheck dispone actualmente de un MVP funcional completo, existen diferentes mejoras previstas para futuras versiones del sistema.

Entre las principales mejoras previstas se encuentran:

- Implementación de autenticación biométrica mediante huella dactilar y reconocimiento facial.
- Generación de informes médicos exportables en formato PDF.
- Mejora del modo offline y sincronización diferida de datos.
- Sistema avanzado de analítica y métricas clínicas.
- Integración con dispositivos wearables y sistemas de salud externos.
- Historial médico ampliado y exportación de tratamientos.
- Mejora del sistema SOS con integración real de geolocalización y emergencias.
- Optimización visual y adaptación para tablets.

Estas mejoras permitirán continuar evolucionando MediCheck hacia una plataforma médica más completa, escalable y preparada para entornos reales.

## 9. Conclusión

MediCheck ha evolucionado desde una idea inicial orientada al seguimiento de tratamientos médicos hacia un MVP funcional completo basado en tecnologías modernas y servicios cloud.

La aplicación implementa una arquitectura modular, persistencia en tiempo real mediante Firebase, control de acceso por roles, validaciones clínicas mediante OpenFDA y un sistema de notificaciones proactivo orientado a mejorar la seguridad del paciente.

El proyecto demuestra la viabilidad técnica y funcional del sistema, así como la capacidad de ampliar la plataforma mediante futuras integraciones y mejoras orientadas a entornos médicos reales.

La estructura implementada permite continuar evolucionando MediCheck hacia una solución más completa, escalable y preparada para escenarios reales de seguimiento sanitario.